

9. CLASES PARA LA LECTURA DE DOCUMENTOS DE INTERCAMBIO DE DATOS.....	1
9.1. Estructura de los ficheros XML.....	1
9.1.1. Declaración XML.....	1
9.1.2. Elementos XML.....	1
9.1.3. Atributos XML.....	2
9.1.4. Contenido XML.....	2
9.2. Escritura de ficheros XML con Java.....	3
9.2. Lectura de ficheros XML con Java.....	6

9. CLASES PARA LA LECTURA DE DOCUMENTOS DE INTERCAMBIO DE DATOS

Existen una gran variedad de tipos de ficheros que se pueden utilizar para intercambiar información entre organizaciones por medios electrónicos, desde simples txt hasta otros muchos más complejos y encriptados. Algunos de los más conocidos actualmente son Json y XML. En nuestro caso nos vamos a centrar en estos últimos.

XML, siglas de lenguaje de marcado extensible, permite definir y almacenar datos de forma compartible. XML admite el intercambio de información entre sistemas de computación, como sitios web, bases de datos y aplicaciones de terceros. Las reglas predefinidas facilitan la transmisión de datos como archivos XML a través de cualquier red, ya que el destinatario puede usar esas reglas para leer los datos de forma precisa y eficiente.

Un archivo de lenguaje de marcado extensible (XML) es un documento basado en texto que se puede guardar con la extensión .xml.

Cualquier archivo XML incluye los siguientes componentes:

9.1. Estructura de los ficheros XML

9.1.1. Declaración XML

Un documento XML comienza con alguna información sobre el propio XML. Por ejemplo, podría mencionar la versión XML que sigue. Esta apertura se denomina declaración XML. A continuación se muestra un ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
```

9.1.2. Elementos XML

Todas las demás etiquetas que se creen en un documento XML se denominan elementos XML. Los elementos XML pueden contener las siguientes características:

- Texto

- Atributos
- Otros elementos

Todos los documentos XML comienzan con una etiqueta principal, que se denomina elemento raíz.

Por ejemplo:

```
<ListaInvitación>
<familia>
  <tía>
    <nombre>Cristina</nombre>
    <nombre>Estefanía</nombre>
  </tía>
</familia>
</ListaInvitación>
```

<ListaInvitación> es el elemento raíz; familia y tía son otros nombres de elementos.

9.1.3. Atributos XML

Los elementos XML pueden tener otros descriptores denominados atributos. Puede definir sus propios nombres de atributos y escribir los valores de los atributos entre comillas, como se muestra a continuación.

```
<edad de la persona="22">
```

9.1.4. Contenido XML

Los datos de los archivos XML también se denominan contenido XML. Por ejemplo, en el archivo XML, es posible que veas datos como este.

```
<amigo>
  <nombre>Carlos</nombre>
  <nombre>Esteban</nombre>
</amigo>
```

Los valores de los datos Carlos y Esteban son el contenido.

9.2. Escritura de ficheros XML con Java

Para poder crear un fichero XML desde nuestro programa vamos a necesitar importar una gran cantidad de librería. A continuación tenemos el listado:

```
java.io.File;
javax.xml.parsers.DocumentBuilder;
javax.xml.parsers.DocumentBuilderFactory;
javax.xml.parsers.ParserConfigurationException;
javax.xml.transform.Result;
javax.xml.transform.Source;
javax.xml.transform.Transformer;
javax.xml.transform.TransformerException;
javax.xml.transform.TransformerFactory;
javax.xml.transform.dom.DOMSource;
javax.xml.transform.stream.StreamResult;
org.w3c.dom.Attr;
org.w3c.dom.DOMImplementation;
org.w3c.dom.Document;
org.w3c.dom.Element;
org.w3c.dom.Text;
```

Gracias a todas estas librerías seremos capaces de crear nuestros propios ficheros XML desde un programa escrito en Java. Veamos el procedimiento:

En primer lugar necesitaremos crear un objeto de tipo DocumentBuilderFactory mediante el método DocumentBuilderFactory.newInstance() y con el objeto resultado vamos a generar un objeto DocumentBuilder usando el método newDocumentBuilder(). Es importante que esto lo introduzcamos dentro de una estructura try...catch, de la siguiente manera.

```
public static void crearFicheroXML(){
    DocumentBuilderFactory factory =
    DocumentBuilderFactory.newInstance();
    DocumentBuilder builder;
    try {
        builder = factory.newDocumentBuilder();
        ...
    } catch (ParserConfigurationException | TransformerException ex) {
        System.out.println(ex.getMessage());
    }
}
```

Después de esto vamos a necesitar crear un objeto DOMImplementation con el método getDOMImplementation. Gracias a este objeto ya podremos crear el documento XML en si con el método createDocument, al que le pasaremos el nombre de nuestro elemento raíz. Los otros dos parámetros los podemos dejar a null. También le indicaremos la versión de XML que vamos a utilizar, en nuestro caso la 1.0.

```
public static void crearFicheroXML(){
    DocumentBuilderFactory factory =
    DocumentBuilderFactory.newInstance();
    DocumentBuilder builder;
    try {
        builder = factory.newDocumentBuilder();
        DOMImplementation implementation =
        builder.getDOMImplementation();

        Document documento = implementation.createDocument(null,
        "elementoRaiz", null);
        documento.setXmlVersion("1.0");
        ...

    } catch (ParserConfigurationException | TransformerException ex) {
        System.out.println(ex.getMessage());
    }
}
```

Una vez que tenemos el documento con el nodo raíz, solo tenemos que ir añadiendo los distintos elementos con sus subelementos, atributos y contenido.

Para esto es importante que siempre tengamos en mente la estructura que deseamos construir dentro del documento.

Vamos a realizar un ejemplo sencillo para obtener una estructura como la que se muestra a continuación.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<elementoRaiz>
    <elementoPadre>
        <elementoHijo1 AtributoHijo1="ValorAtributoHijo1" />
        <elementoHijo2>textoElementoHijo2</elementoHijo2>
```

```
<elementoHijo3>textoElementoHijo3</elementoHijo3>
</elementoPadre>
</elementoRaiz>
```

Creamos el elemento padre...

```
Element elementoPadre = documento.createElement("elementoPadre");
```

Creamos el elemento hijo 1 y le asignamos el atributo y el valor...

```
Element elemento1 = documento.createElement("elementoHijo1");
elemento1.setAttribute("AtributoHijo1", "ValorAtributoHijo1");
```

Creamos los elementos hijos 2 y 3, sus contenidos y unimos cada uno al que le corresponde...

```
Element elemento2 = documento.createElement("elementoHijo2");
Text textoElemento2 = documento.createTextNode("textoElementoHijo2");
elemento2.appendChild(textoElemento2);
```

```
Element elemento3 = documento.createElement("elemento3");
Text textoElemento3 = documento.createTextNode("textoElementoHijo3");
elemento3.appendChild(textoElemento3);
```

Después, añadimos los nodos hijos al nodo padre...

```
elementoPadre.appendChild(elemento1);
elementoPadre.appendChild(elemento2);
elementoPadre.appendChild(elemento3);
```

Añadimos el nodo padre al nodo raíz...

```
documento.getDocumentElement().appendChild(elementoPadre);
```

Finalmente, damos un nombre al nuevo fichero e introducimos el documento XML que hemos generado en dicho fichero.

```
Result result = new StreamResult(new File("elementos.xml"));
Source source = new DOMSource(documento);
```

```
Transformer transformer =  
TransformerFactory.newInstance().newTransformer();  
transformer.transform(source, result);
```

9.3. Lectura de ficheros XML con Java

Para leer un fichero XML, y cargarlo en memoria, a través de un programa codificado en Java también necesitaremos hacer uso de varias librerías:

```
java.io.File;  
java.io.IOException;  
javax.xml.parsers.DocumentBuilder;  
javax.xml.parsers.DocumentBuilderFactory;  
javax.xml.parsers.ParserConfigurationException;  
org.w3c.dom.Document;  
org.w3c.dom.Element;  
org.w3c.dom.NamedNodeMap;  
org.w3c.dom.Node;  
org.w3c.dom.NodeList;  
org.xml.sax.SAXException;
```

Para leer los documentos XML nos vamos a ayudar de los nombres de las etiquetas de los nodos, y a partir de ahí leeremos sus nodos hijos.

Veamos cómo leeríamos el fichero que hemos creado antes:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>  
<elementoRaiz>  
  <elementoPadre>  
    <elementoHijo1 AtributoHijo1="ValorAtributoHijo1" />  
    <elementoHijo2>textoElementoHijo2</elementoHijo2>  
    <elementoHijo3>textoElementoHijo3</elementoHijo3>  
  </elementoPadre>  
</elementoRaiz>
```

Lo primero que vamos a hacer, al igual que con la escritura es crear los objetos DocumentBuilderFactory y DocumentBuilder.

```
public static void crearFicheroXML(){  
  DocumentBuilderFactory factory =  
DocumentBuilderFactory.newInstance();  
  DocumentBuilder builder;  
  try {  
    builder = factory.newDocumentBuilder();
```

```

    ...

    } catch (ParserConfigurationException | SAXException | IOException
ex){
        System.out.println(ex.getMessage());
    }
}

```

Después abrimos el fichero xml que vamos a leer...

```
Document documento = builder.parse(new File("elementos.xml"));
```

Buscamos todos los nodos cuya etiqueta sea elementoPadre...

```
NodeList listaPadres = documento.getElementsByTagName("elementoPadre");
```

En nuestro caso solo hay uno, pero al almacenarlo en una lista podemos almacenar un número indeterminado de ellos y luego ir examinando uno a uno.

A continuación, por cada uno de los nodos que nos ha devuelto, vamos a comprobar que es un nodo del tipo Element, y si lo es extraeremos sus hijos.

```

for(int i = 0; i < listaPadres.getLength(); i++){
    Node nodo = listaPadres.item(i);
    if(nodo.getNodeType() == Node.ELEMENT_NODE){
        Element e = (Element) nodo;
        System.out.println("Elemento: " + nodo.getNodeName());
        NodeList hijos = e.getChildNodes();
        ...
    }
}

```

Como sabemos que unos elementos tienen atributos y otros contenido, vamos a preguntar a cada nodo hijo por el número de atributos que tiene. Si tiene más de 0 sabemos que es un elementoHijo1, si no, es de lo otros.

```

for (int j = 0; j < hijos.getLength(); j++){
    Node hijo = hijos.item(j);
    if(hijo.getNodeType() == Node.ELEMENT_NODE){
        NamedNodeMap atributos = hijo.getAttributes();
        if(atributos.getLength() > 0){
            Node atributo = atributos.getNamedItem("AtributoHijo1");

```

```
        System.out.println("NodoHijo: " + hijo.getNodeName() + ",  
Atributo: " + atributo.getNodeValue());  
    }  
    else{  
        System.out.println("NodoHijo: " + hijo.getNodeName() + ",  
Valor: " + hijo.getTextContent());  
    }  
  
    }  
}
```

El resultado que nos devolvería por consola al ejecutar este procedimiento de lectura sería el siguiente:

Elemento: elementoPadre

NodoHijo: elementoHijo1, Atributo: ValorAtributoHijo1

NodoHijo: elementoHijo2, Valor: textoElementoHijo2

NodoHijo: elementoHijo3, Valor: textoElementoHijo3

Vamos a practicar creando y leyendo ficheros XML.

